

Software Requirements

Specification (SRS)

Project: Taste Matcher
Author: Srirama Prabhav R.
Date: May 4, 2026

1. Introduction

Taste Matcher v3.1 is a backend-first Node.js application designed to ingest Letterboxd CSV exports, cross-reference them with the TMDb API, and generate a personalized, mathematically ranked watchlist based on a multi-dimensional taste profile.

The application runs entirely on the user's local machine — no cloud service, no database server, no third-party account beyond a free TMDb API key. All state is persisted via flat JSON files in a local `/cache/` directory.

2. User Roles

2.1 End-User

The primary (and sole) user of the system. Responsibilities include:

- Exporting `ratings.csv` and `watchlist.csv` from Letterboxd
- Configuring the TMDb API key via `.env` or the Electron setup wizard
- Interacting with the UI to view rankings, apply filters, and add manual ratings
- Exporting the final ranked list as a Letterboxd-compatible CSV

3. Functional Requirements

ID	Requirement	Implementation Notes
RE Q-0 1	Data Ingestion: The system must parse standard Letterboxd <code>ratings.csv</code> and <code>watchlist.csv</code> exports.	Handled by <code>loadRatingsFromCsv()</code> and <code>loadWatchlistFromCsv()</code> in <code>server.js</code> . Auto-detects legacy and v7 header formats using regex column matching.
RE Q-0 2	API Integration: The system must fetch movie/TV metadata (genres, director/writer credits, keywords, countries, decades, collections) via the TMDb REST API.	Handled by <code>getMovieDetailsForItem()</code> . Supports both movie and TV search. Calls <code>/movie/{id}</code> , <code>/tv/{id}</code> , <code>/credits</code> , and <code>/keywords</code> endpoints in parallel via <code>Promise.all()</code> .

ID	Requirement	Implementation Notes
RE Q-0 3	Scoring Engine: The system must build a Bayesian-smoothed taste profile and assign a predicted affinity score (0.00–1.00) to each unrated watchlist film.	buildTasteModel() constructs nine dimension profiles with SMOOTH_K=5. scoreItem() computes the composite score using activeWeights. weightsFromPriorityRanking() converts user priority order to geometric-decay weights.
RE Q-0 4	Duplicate Prevention: The system must reject duplicate manual entries using a triple-validation check.	add-rating and add-to-watchlist endpoints check: (1) Letterboxd URL match, (2) Title + Release Year match, (3) TMDb ID match. Any match triggers a 409 response.
RE Q-0 5	Data Export: The system must export the ranked list in strict Letterboxd list v7 CSV format, streamed directly to the browser.	The GET /api/export-ranked-watchlist endpoint re-scores all candidates using the shared scoreItem(), generates a v7-compliant CSV (with list header, metadata row, and position-indexed film rows), and streams it via res.send() with Content-Disposition: attachment.
RE Q-0 6	Live Mutation System: All add/update/remove operations must update the in-memory state and invalidate derived caches without requiring a server restart.	All mutation endpoints call invalidateAllDerivedCaches() or invalidateWatchlistCaches() and saveDataStateToDisk(). State survives restarts via data_state.json.
RE Q-0 7	Rewatch Ranking: Films appearing in both the ratings and watchlist must be scored separately using a rating-anchored formula.	rewatchScore = userRatingNorm × (0.70 + 0.30 × modelScore). Prevents the model from overriding explicit user judgments on seen films.
RE Q-0 8	User-Configurable Weights: Users must be able to reorder the nine scoring dimensions without exposing numeric weights.	POST /api/user-weights accepts a ranked array of dimension keys. weightsFromPriorityRanking() converts it to geometric-decay weights stored in user_weights.json.

4. Non-Functional Requirements

4.1 Performance — Caching

To operate within TMDb's free-tier rate limits, the system must cache all API responses locally:

- `tmdb_cache.json`: Raw TMDb metadata. Once written, never re-fetched.
- `derived_cache.json`: Computed taste model, ranked recommendations, overlap details, and rewatch ranking.
- `data_state.json`: In-memory ratings, watchlist, and overlap arrays — persisted so mutations survive restarts.
- `user_weights.json`: User's active weight distribution.
- `hidden_rewatches.json`: Set of hidden rewatch URLs.

4.2 Environment

The application must run locally without a dedicated database server. Specific constraints:

- Runtime: Node.js 18+
- No external database — all state via flat JSON files in /cache/
- All configuration via .env (plain-node mode) or Electron IPC (desktop mode)
- Must support cross-platform Electron packaging: Windows (NSIS installer), macOS, Linux
- Electron user data stored in OS-standard directories: %APPDATA% (Windows), ~/Library/Application Support (macOS), ~/.config (Linux)

4.3 Reliability

- Derived cache entries missing ratedItemsMeta are detected on load and discarded, forcing a clean rebuild
- All mutation endpoints return structured JSON errors with appropriate HTTP status codes
- Failed TMDb resolutions are tracked in failedItems and surfaced in the UI for manual resolution

4.4 Security

- The Electron renderer is sandboxed: contextIsolation: true, nodeIntegration: false
- File picker and CSV install operations are exposed only via a validated IPC bridge (preload.js)
- TMDb API key is stored in config.json in the OS user-data directory, never in the app bundle